

React Interview Prep

Topic 8 — useRef: DOM Access, Mutable Values & forwardRef

BEGINNER → INTERMEDIATE → EXPERT

■ Covers

useRef · DOM access · Mutable storage · forwardRef · useImperativeHandle · Rules

■ Series

React Interview Prep — Session 8

by Akshay Chavhan • React Interview Series

1. What is useRef?

`useRef` returns a **mutable object** that persists across renders. Changing it never triggers a re-render — React doesn't know or care.

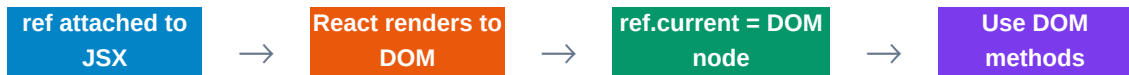
```
const ref = useRef(initialValue);
// ref looks like this:
{ current: initialValue }
// always accessed via .current
```

■ ***useRef has TWO use cases: 1. Access real DOM elements directly 2. Store mutable values without triggering re-render***

2. Use Case 1 — DOM Access

Attach ref to any JSX element — React sets `.current` to the actual DOM node after render.

```
function InputFocus() {
  const inputRef = useRef(null); // null before mount
  function handleClick() {
    inputRef.current.focus(); // real DOM method ■
  }
  return (
    <>
    <input ref={inputRef} type='text' />
    <button onClick={handleClick}>Focus</button>
    </>
  );
}
```



```
// Common DOM operations via ref:
ref.current.focus() // focus input
ref.current.scrollIntoView() // scroll to element
ref.current.value // read input value
ref.current.getBoundingClientRect() // get dimensions
```

3. Use Case 2 — Mutable Storage Without Re-render

Store values that change but don't need to update the UI — like timer IDs, previous values, or flags.

```
function Timer() {
  const [count, setCount] = useState(0);
  const timerRef = useRef(null); // store timer ID silently
  function start() {
    timerRef.current = setInterval(() => {
      setCount(c => c + 1);
    }, 1000);
  }
  function stop() {
    clearInterval(timerRef.current); // access stored ID ■
  }
}
```

■ If you stored `timerRef` in `useState`, it would trigger a re-render on every start/stop. `useRef` stores it silently — zero re-renders.

4. `useRef` vs `useState`

	<code>useState</code>	<code>useRef</code>
Triggers re-render	■ Yes	■ No
Persists across renders	■ Yes	■ Yes
Access	value directly	<code>.current</code>
Update is synchronous	■ No (batched)	■ Yes (immediate)
Use when	UI must reflect change	Silent storage / DOM access
Show in UI	■ Yes	■ Avoid — UI won't update

■ Wrong

```
// ■ ref for UI value
const count = useRef(0);
count.current += 1;
// UI never updates!
return {count.current};
```

■ Correct

```
// ■ state for UI value
const [count, setCount]
= useState(0);
setCount(c => c + 1);
return {count}; // ■
```

5. Track Previous State Value

A classic useRef pattern — store the previous render's value.

```
function App() {
  const [name, setName] = useState('Akshay');
  const prevName = useRef(name);
  useEffect(() => {
    prevName.current = name; // updates AFTER render
  }, [name]);
  return (
    <div>
      <p>Current: {name}</p>
      <p>Previous: {prevName.current}</p>
    </div>
  );
}
```

■ **Effect runs AFTER render** — so `prevName.current` holds the old value during current render, then updates to new value after. Perfect previous-value tracker.

6. forwardRef — Pass Ref to Child Component

By default, ref doesn't work on custom components. `forwardRef` solves this.

■ Wrong

```
// ■ ref on custom component
// ref.current = null always!
function Parent() {
  const ref = useRef(null);
  return ;
  // ref is null — won't work ■
}
```

■ Correct

```
// ■ wrap child in forwardRef
const CustomInput = forwardRef(
  (props, ref) => (
    )
  );
// ref.current = DOM input ■
```

■ **`forwardRef` passes the ref as a second argument to the child component, which then forwards it to the real DOM node inside.**

7. useImperativeHandle — Control What Ref Exposes

Limit what the parent can do with the child's ref — expose only specific methods.

```
const CustomInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(), // ■ allowed
    clear: () => { inputRef.current.value = ''; } // ■ allowed
  }));
  // parent CANNOT access .style, .id etc ■
  return <input ref={inputRef} {...props} />;
});
// Parent usage:
```

```
ref.current.focus(); // ■  
ref.current.clear(); // ■  
ref.current.style; // ■ undefined – not exposed
```

■ *useImperativeHandle gives child control over its own API — parent can only call what child allows.*

8. Key Rules of useRef

Rule	Detail
Never read/write ref.current during render	Side effect during render = impure component. Do it in useEffect or event handlers only.
Never use ref for values that show in UI	Ref changes don't trigger re-render — UI stays stale. Use useState instead.
ref.current is always synchronous	Unlike state updates (batched), ref.current = value is immediate and always up to date.
ref on custom component = null by default	Must use forwardRef to pass ref through a custom component to a DOM node.
Initial value only used on first render	useRef(initialValue) — the initial value is only used on mount, same as useState.

9. Interview Q&A;

Q

Beginner

Q1. What is useRef and what are its two main uses?

→ useRef returns a mutable object { current: value } that persists across renders. First use: accessing real DOM nodes directly by attaching ref to JSX elements. Second use: storing mutable values like timer IDs or flags without triggering re-renders.

Q

Beginner

Q2. What is the difference between useRef and useState?

→ useState triggers a re-render when updated and the new value is available on the next render. useRef never triggers re-renders, updates synchronously, and is accessed via .current. Use useState when UI must reflect the change. Use useRef for silent storage or DOM access.

Q

Intermediate

Q3. Why can't you pass ref to a custom component directly?

→ React doesn't forward the ref prop to custom components by default — it's a reserved prop. You need to wrap the child in forwardRef, which receives the ref as a second argument and forwards it to the actual DOM node inside.

Q

Intermediate

Q4. What is forwardRef?

→ forwardRef is a React API that lets a parent component pass a ref through a custom child to a real DOM node inside it. Without forwardRef, ref on a custom component is always null.

Q

Intermediate

Q5. Can you store a previous state value using useRef?

→ Yes — store the value in useEffect that runs when state changes. The effect runs after render, so the ref holds the previous value during the current render and updates to the new value after. This is a classic pattern for tracking previous props or state.

Q

Expert

Q6. What is useImperativeHandle and when would you use it?

→ useImperativeHandle works with forwardRef to control what the parent can access via ref. Instead of exposing the entire DOM node, you expose only specific methods — giving the child control over its own API. Use it in reusable component libraries to prevent misuse of internal DOM.

Q

Expert

Q7. Why should you not read or write ref.current during render?

→ Refs are mutable and changes aren't tracked by React. Reading or writing during render makes the component impure — same inputs could produce different outputs. In Strict Mode, React renders twice, which would cause double mutation bugs. Always read/write refs inside useEffect or event handlers.

